

MarkChem：面向大模型化学回答的可编译分子标记语言

何梓源

2026 年 7 月 2 日

摘要

本文报告一个面向化学结构表达与网页渲染的小型编译系统。系统定义了一种文本 DSL，将大模型或人工给出的分子片段表达式编译为显式的分子连接图 IR，并进一步渲染为 SVG 键线式。本文目前重点描述技术模块，包括语言语法、语义设计、AST 到 IR 的翻译、前后端划分以及 C++ 实现。研究动机、实验结果和讨论将在后续版本中补充。

1 动机

- 内嵌 LaTeX 公式的 Markdown 是大模型输出的事实标准，其对数学/物理公式有着很好的支持，不仅可以由 AI 生成，人类经过简单学习也可以看懂并编辑。
- 目前现的对化学领域常见的结构表达方式大致有以下几种

例子	优势	劣势
SVG	直观、生成成本较低	需要模型多模态能力、编辑困难、丢失了高级语义
SMILES	化学领域数据库常用标准	人类难懂、编辑困难
InChI	具有唯一性，索引方便	语法限制性强，图到文转化困难
ToolCall	直接接入专业软件、正确稳定	现有支持不多，不利于构建完整生态

- 于是我希望提出一种 AI 可生成、人类可手写，检查，局部编辑、适合网页内嵌的 DSL。

2 系统概览

系统将分子结构表达建模为一次小型编译过程：

my-Chem source $\rightarrow$ Token stream $\rightarrow$ AST $\rightarrow$ IR graph $\rightarrow$ SVG.

前端负责识别和解析源语言，后端负责语义翻译与渲染。整体实现由一个 C++ 编译核心库和若干入口程序组成：

- 词法分析器**：将输入字符串切分为括号、分隔符、数字、关键字等 token。
- 递归下降 Parser**：根据文法构造 AST，保留 CPO、接口、核心片段和取代基结构。

- **语义翻译器**: 把 AST 翻译为无序分子连接图 IR, 并完成接口合法性、端基、耦合、原子替换等语义检查。
- **SVG 渲染后端**: 优先将 IR 降低为 RDKit 分子对象并调用 RDKit 2D depiction; 当 RDKit 不可用时使用内置 SVG fallback。
- **浏览器插件与本地服务**: 网页中的 `<my-Chem>...</my-Chem>` 内容由插件提取, 发送给本地 C++ 服务, 返回 SVG 后替换原标签。

3 源语言设计

3.1 CPO 片段

源语言的基本单位是 CPO (chemical phrase object)。一个完整输入由一个或多个 CPO 串联组成, 每个 CPO 以分号结束:

```
CPD      := CPO+
CPO      := INTERFACE CORE "," SUBS INTERFACE ";"
INTERFACE := "(" POSES ")" | "{" POSES "}"
POSES    := digit ("," digit)*
```

一个 CPO 包含四个部分: 左接口、核心片段、取代基列表、右接口。例如:

```
(0)Ph,1-OH,(0);
(0)NH2CC,4-Me,5-=O,5-OH,(0);
(0)6R,{4,5};{1,2}6R,(0);
```

接口中的编号均为当前 CPO 内部的局部原子编号。数字 0 表示空接口, 不参与跨 CPO 连接。

3.2 核心关键字

核心片段由关键字序列构成。当前实现支持如下关键字:

类别	关键字	含义
单原子	C P O N S F Cl Br I	对应元素原子
常用取代基	Me Ac NH2 OH SH OMe OPh =O	预定义官能团或片段
常见环系	Ph Im Ind Pyr	常用环片段
参数化骨架	nR nL	n 元碳环或 n 原子碳链
键阶修饰	= #	双键或三键修饰符
掺杂修饰	~X	X 杂原子进入环系

3.3 CPO 间接口: 连边与耦合

接口括号类型具有明确语义:

- 圆括号 (p1,p2) 表示直接连边接口。相邻 CPO 的对应接口原子按位置逐对相连。
- 花括号 {p1,p2} 表示共用原子接口。相邻 CPO 的对应接口原子被视为同一组原子，用于耦合环。

例如：

```
(0)2L,(1,2);(1,2)2L,(0);
```

该表达式将第一个 CPO 的 1、2 号原子分别与第二个 CPO 的 1、2 号原子直接相连，形成两条跨 CPO 键。相反：

```
(0)6R,{4,5};{1,2}6R,(0);
```

表示第二个六元环的 1、2 号原子与第一个六元环的 4、5 号原子共用，从而构造耦合环。相邻接口必须使用相同括号类型，否则语义翻译阶段报错。旧规则中“两个非零接口自动耦合”的推断已被删除，因此 (4,5);(1,2) 现在表示两条直接跨片段键，而不是耦合。

3.4 取代基与原子替换

CPO 的 SUBS 部分描述当前核心上的局部修改：

p-X,	attach: 在 p 号原子外接 X
p1,p2-X,	attach many: 在多个位置外接 X
p=,	升级 p 到 defaultNext[p] 的内部键为双键
p= <u>0</u> ,	在 p 号原子上外接双键氧
p [~] X,	replace: 用 X 的入口原子替代 p 号原子
p-[CPO],	外接结构化 CPO 片段
p [~] [CPO],	用结构化 CPO 的入口原子替代 p 号原子

p-X 和 p[~]X 的区别是语言设计中的关键点。前者在原子 p 外新增取代基，原子 p 保持不变；后者将 p 号原子本身替换为 X 的入口原子，并保留 p 原有的键。例如：

```
(0)6R,1-0,(0);    # 在 1 号碳上外接氧
(0)6R,1-~0,(0);    # 将 1 号碳替换为氧杂原子
```

结构化取代基使用方括号包围一个完整 CPO：

```
(0)6R,3-[(1)COH,(0)],(0);
```

组取代基要求非零左接口唯一，从而确定该组连接到母体的位置。

4 词法与语法分析实现

4.1 词法分析

词法分析器按最长匹配规则识别关键字。由于 OMe、OPh、OH 与单原子 O 共享前缀，关键字表将长关键字排在短关键字之前，避免将 OMe 错误拆成 O 与标识符。

当前 token 类型包括：

词法阶段也承担部分早期错误检查。例如当读到数字后紧跟大写字母且不是 R 或 L 时，直接拒绝 5O 这类旧补丁语法，并提示用户改用 5-[~]O。

Token	例子	用途
Sep	,	分隔接口编号或取代项
Semi	;	结束一个 CPO
LParen/RParen	(,)	直接连边接口
LBrace/RBrace	{, }	共用原子接口
LStr/RStr	[,]	结构化取代基
Keyword	Ph, =O	化学片段关键字
Digit	0, 12	局部位置编号
Haf	-	位置与取代基之间的连接标记
Caret	^	原子替换标记

4.2 递归下降 Parser

Parser 使用手写递归下降实现。每个非终结符对应一个解析函数, 例如 `parseCpd`、`parseCpo`、`parseInterface`、`parseSubs`、`parseCore`。该选择的原因是: 语言规模小、文法结构清晰, 并且手写 parser 便于给出面向 DSL 用户的错误信息。

AST 的核心数据结构包括:

- **Cpd**: 完整程序, 由多个 CPO 组成;
- **Cpo**: 左接口、核心、取代基、右接口;
- **Interface**: 接口编号与接口类型 `Connect/Fuse`;
- **Core**: 关键字序列;
- **Subs**: 取代基链表, 记录 `attach`、`replace`、`group` 等信息。

语法分析只保证输入符合上下文无关结构。诸如接口编号是否存在、耦合原子类型是否匹配、端基是否继续延展等问题留给语义翻译阶段检查。

5 IR 与语义设计

5.1 IR 节点

IR 是显式无序连接图。每个节点表示一个原子:

```
ATOM(self_id, conn1, conn2, conn3, conn4, conn5, conn6)
```

`self_id` 是唯一原子编号; `conn_i` 是相邻原子编号, 0 表示空连接槽。键阶通过重复邻接点表示: 单键出现一次, 双键出现两次, 三键出现三次。例如 `0(3,2,2,0,0,0,0)` 表示 3 号氧与 2 号原子形成双键。

5.2 Fragment 属性

AST 到 IR 的翻译以 `Fragment` 为核心综合属性：

```
Fragment {
    vector<IrNode> ir;
    int entry;
    optional<int> exit;
    int entryBond;
    vector<int> interfaceL, interfaceR;
    InterfaceKind interfaceLKind, interfaceRKind;
    map<int,int> defaultNext;
}
```

其中：

- `entry` 表示片段被外部接入时使用的入口；
- `exit` 表示线性拼接时的出口，`nullopt` 表示端基，不可继续作为 CORE 尾部延展；
- `entryBond` 表示外部连接到入口时使用的键阶；
- `interfaceL/interfaceR` 保存 CPO 左右接口编号；
- `interfaceLKind/interfaceRKind` 保存接口语义，即直接连边或共用原子；
- `defaultNext` 保存 `p==` 这类内部升键操作的默认目标。

5.3 关键语义操作

翻译器定义若干基本图操作：

- `shift(F, k)`：将片段中所有非零 id 和连接编号平移 k ，避免合并时编号冲突；
- `appendIr`：合并两个不冲突的 IR 节点集合；
- `link(a,b,order)`：插入指定键阶的双向连接；
- `ensureBond`：将已有键提升到指定键阶；
- `rewrite(alias)`：在耦合或替换时重写原子编号。

这些操作共同保证 IR 始终保持双向连接一致性，并在连接槽不足时报告 `valence overflow`。

6 AST 到 IR 的翻译

6.1 核心片段翻译

每个关键字首先由 `Keyword.cpp` 翻译为基本片段。例如：

- C、O、N 等单原子关键字翻译为单节点片段；

- Me、OH、NH₂ 翻译为预定义官能团；
- nR 翻译为 n 元环，并建立环上 defaultNext；
- nL 翻译为 n 原子链；
- = 和 # 不产生独立节点，而是作为后续连接的键阶修饰。

当 CORE 包含多个关键字时，翻译器从左到右合并片段。若左侧片段 `exit = nullopt`，说明该片段是端基，不能继续作为线性 CORE 的左侧延展，此时翻译器报错。

6.2 取代基翻译

对于普通取代 $p-X$ ，翻译器执行：

1. 翻译 X 为片段 S ；
2. 将 S 的编号平移到不与母体冲突；
3. 找到 S 的入口；
4. 将母体 p 与入口按 `entryBond` 相连；
5. 合并 S 的 IR 和 `defaultNext`。

对于原子替换 $p-\hat{X}$ ，翻译器不新增一条外部键，而是把 X 的入口原子映射到母体 p ：

1. 要求 p 在母体中存在，且 X 的 IR 非空；
2. 翻译并平移 X ；
3. 选取 X 的入口原子；
4. 将母体 p 的元素类型替换为入口原子的元素类型；
5. 保留 p 的原有连接；
6. 将 X 中除入口外的节点和内部键合并到母体。

该设计使氧杂、氮杂、硫杂等环内原子替换可以统一写成 $p-\hat{O}$ 、 $p-\hat{N}$ 、 $p-\hat{S}$ ，避免引入 50 这类临时补丁语法。

6.3 CPO 间连接与耦合

相邻 CPO 的组合由左片段右接口和右片段左接口决定。若二者括号类型不同，翻译器报错。

直接连边。 当接口类型为 `Connect` 时，要求两侧接口长度相等且不含 0。翻译器平移右片段编号，合并 IR，并对每一对对应接口原子调用 `link`：

$$\forall i, \quad \text{link}(A.\text{interfaceR}_i, B.\text{interfaceL}_i, B.\text{entryBond}).$$

共用原子稠合。 当接口类型为 `Fuse` 时，要求两侧各有两个非零接口。翻译器构造别名映射：

$$B.\text{interface}L_1 \mapsto A.\text{interface}R_1, \quad B.\text{interface}L_2 \mapsto A.\text{interface}R_2.$$

随后将右片段所有节点与边通过该映射重写并合并。若同一个 IR id 对应的原子类型不一致，例如一个是 `N` 而另一个是 `C`，则报告 “Fuse atom mismatch”。这可以阻止不合理的稠合表达式静默生成错误结构。

7 渲染后端

7.1 IR 到分子对象

系统将 IR 降低为 `RDKit::RWMol`：

1. 根据 IR 节点元素类型创建 `RDKit Atom`；
2. 根据 IR 边集合创建 `RDKit Bond`；
3. 调用 `updatePropertyCache(false)`；
4. 调用 `RDKit 2D 坐标生成`；
5. 使用 `MolDraw2DSVG` 输出 SVG。

`RDKit` 后端的优点是布局质量高、能处理常见化学绘图约定；缺点是依赖外部库。项目的 `CMake` 配置中对 `RDKit` 采用手动库发现，以规避部分系统包导出的 `CMake target` 引用缺失 `NumPy` 或 `Cairo` 路径的问题。由于化学物质的极度多样性，后端需要处理大量实际上与编译无关的特例，所以这里引入避免重复工作。

8 使用方式

项目将可复用逻辑封装在 `compiler_core` 静态库中。其上有多个入口：

- `tokenizer`：命令行词法调试工具；
- `parser`：命令行完整编译与渲染工具，会打印 token、AST、IR 和 SVG；
- `chem_http_server`：监听本地端口 `127.0.0.1:17654`，提供 `POST /render`；
- `chem_native_host`：Chrome/Edge native messaging 备用入口。

浏览器插件属于前端外壳。其任务是在网页 DOM 中识别 `<my-Chem>` 标签或被 HTML 转义显示的 `my-Chem` 文本，提取 DSL 内容，通过 HTTP 调用本地 C++ 服务，并用返回的 SVG 替换原位置。插件侧不实现化学语义，避免前后端重复逻辑；所有语法和语义错误均由后端统一返回。对于单个标签解析失败的情况，插件应跳过该标签并继续处理其它标签，以保证页面鲁棒性。后两部分与编译无关，且受限于个人前端水平，实现效果不佳

9 表达能力与 Demo

- 任何物质通过足够多次的切断都变成链状骨架进而被表达，但是只是直观性可能会损失。
- 大模型具有一定生成此 DSL 的能力，只通过对话内的提示（提示词见仓库）加上一些迭代即可完成较复杂的绘图工作，如果使用 skill 可能可以达到更好的效果。<https://chatgpt.com/share/6a46a5a1-6da0-83e8-98e6-7df919d9aca1>。
- 额外提供了一些手写时的语法糖，例如无需写分子前端和后端的 (0) 以及接口前方的,'
- 对于简单的分子，大模型很容易写出 DSL，例如对于所有构成蛋白质的氨基酸：

9.1 简单分子示例

下面的 SVG 图由 `test_cases_amino_acids.tsv` 渲染得到。tsv 由 AI 生成。

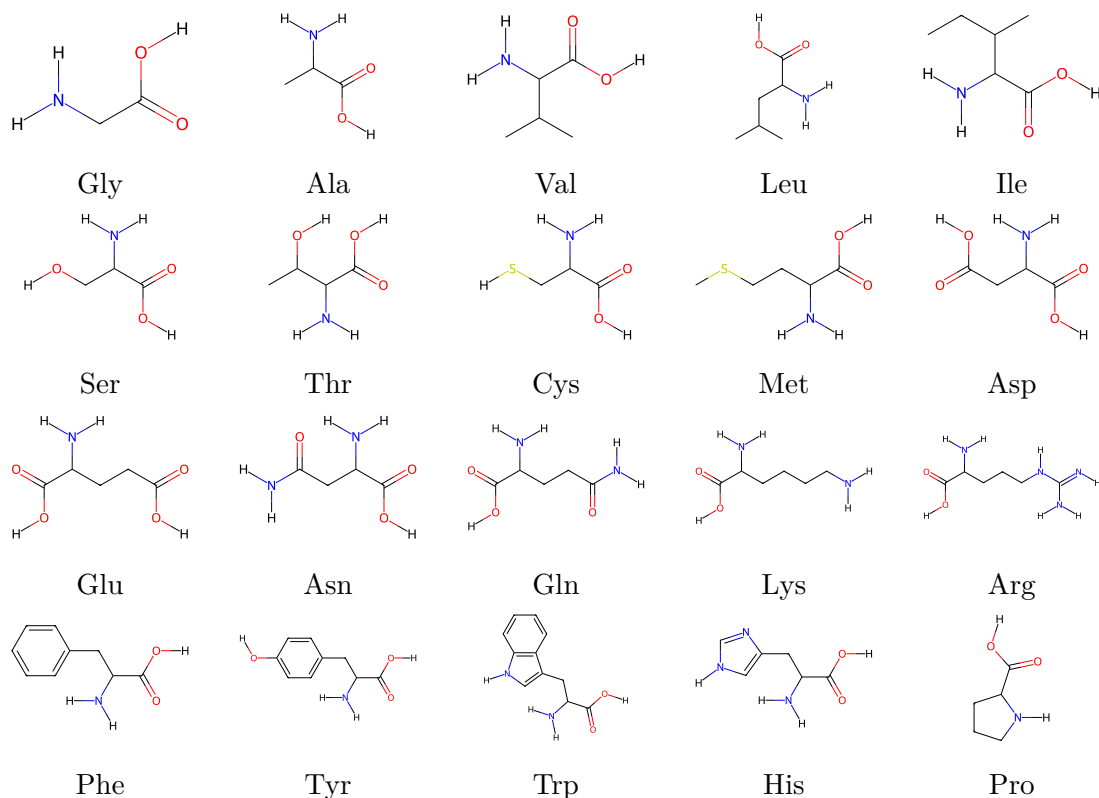
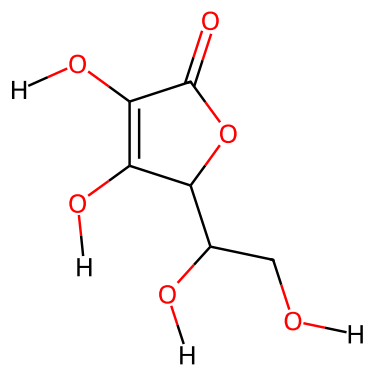


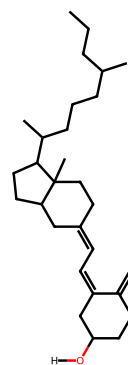
图 1: MarkChem 渲染的氨基酸简单示例。

9.2 复杂分子示例

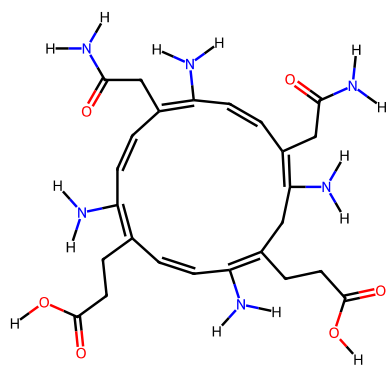
下面的 SVG 图由 `test_cases_complex.tsv` 渲染得到。所有图片均使用仓库相对路径 `output_complex/`。由此可见，对于较为复杂的分子 AI 无法直接生成其 MarkChem



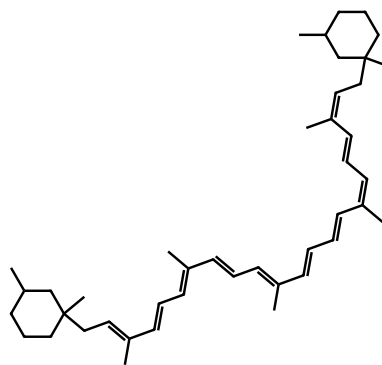
维生素 C



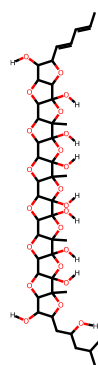
维生素 D3



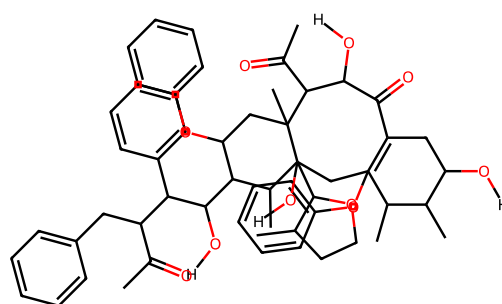
维生素 B12 (生成结构错误)



胡萝卜素

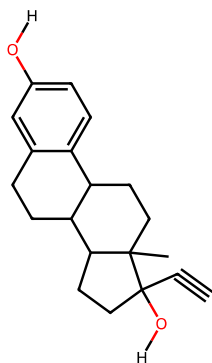


雪卡毒素 (完全错误)

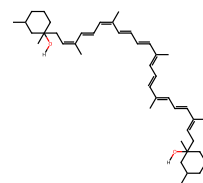


紫杉醇近似 (完全错误)

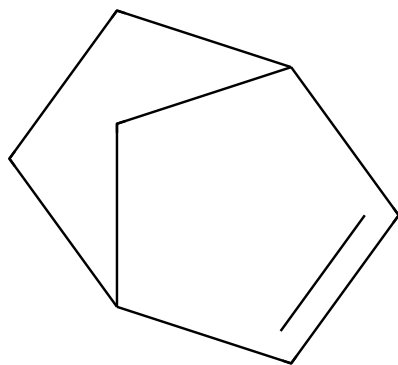
图 2: MarkChem 渲染的复杂分子示例。



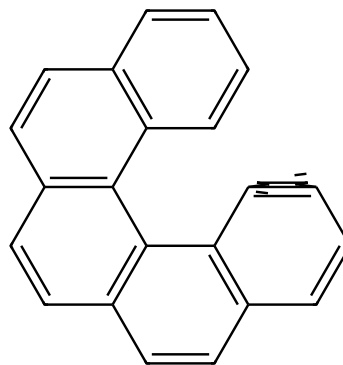
乙炔雌二醇



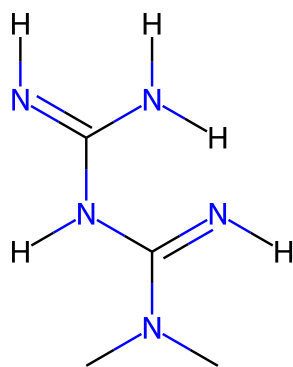
叶黄素（两侧环上取代基错误）



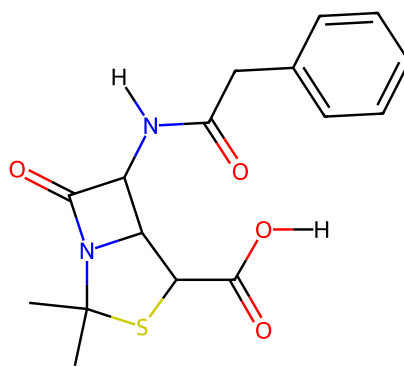
降冰片烯



六螺苯近似（渲染器问题，最后一个环存在压盖）



二甲双胍



青霉素 G 近似

图 3: MarkChem 渲染的复杂分子示例-续表。

10 更多有待完成的工作

1. 整合多个渲染的分子式构成方程式或合成路线图
 2. 支持原子的标号、立体化学
 3. 完善网页插件...
 4. 做成和 md 兼容的形式（尝试把我的模块整合到标准 md 编译器里失败了）
- ...

11 代码仓库与环境配置

代码位于仓库 https://github.com/lambda-z-miu/compiler_project 需配置 RDKit 环境，推荐在 Linux 下进行，作为参考本地开发时使用的 WSL Ubuntu 24.

12 AI 使用说明、工作量

- 前端约 1.3k LOC，人工完成词法、文法和翻译过程的设计，AI 完成代码，略有改动。
- 后端约 500 行，几乎全由 AI 完成，人工只决策了调用 RDKit 这件事。
- 使用 codex + GPT5.5